

# EXPERIMENT - 33

## 1. Problem Statement

Write a C program to perform the following menu-driven operations using separate functions:

- Read three variable-length 2D integer arrays A, B and C, each of dimensions  $(m \times n)$ , and verify the associative property of matrix addition:  $(A+B)+C = A+(B+C)$ .
- Read three variable-length 2D integer arrays with the dimensions A  $(m \times n)$ , B  $(n \times p)$ , C  $(p \times q)$ , and verify the associative property of matrix multiplication:  $(A \times B) \times C = A \times (B \times C)$ .
- Exit.

For each operation the matrices obtained on both sides of the equation are displayed, and the program states whether the associative property is verified.

## 2. Algorithm

### Main Program

**Step 1:** START

**Step 2:** Display the menu and read choice.

**Step 3:** Using a switch statement on choice:

    If choice is 1, call additionAssociative().

    If choice is 2, call multiplicationAssociative().

    If choice is 3, display "Exiting the program."

**Step 4:** Repeat from Step 2 while choice is not equal to 3.

**Step 5:** STOP

### Function additionAssociative()

**Step 1:** Read m and n, and declare the variable length arrays a, b, c, left and right, all of size  $m \times n$ .

**Step 2:** Read the elements of A, then of B, then of C.

**Step 3:** Repeat for  $i = 0$  to  $m - 1$  and for  $j = 0$  to  $n - 1$ :

    Set  $\text{left}[i][j] = (a[i][j] + b[i][j]) + c[i][j]$ .

    Set  $\text{right}[i][j] = a[i][j] + (b[i][j] + c[i][j])$ .

**Step 4:** Display  $(A + B) + C$  and  $A + (B + C)$ .

**Step 5:** If the two are equal place by place, display that the property is verified.

**Step 6:** Return to the calling function.

### Function multiplicationAssociative()

**Step 1:** Read m, n, p and q, and declare a of size  $m \times n$ , b of size  $n \times p$ , c of size  $p \times q$ , ab of size  $m \times p$ , bc of size  $n \times q$ , and left and right of size  $m \times q$ .

**Step 2:** Read the elements of A, then of B, then of C.

**Step 3:** Find  $ab = A \times B$ , then  $\text{left} = ab \times C$ .

**Step 4:** Find  $bc = B \times C$ , then  $\text{right} = A \times bc$ .

**Step 5:** Display  $(A \times B) \times C$  and  $A \times (B \times C)$ .

**Step 6:** If the two are equal place by place, display that the property is verified.

**Step 7:** Return to the calling function.

### Function multiply()

**Step 1:** Repeat for  $i = 0$  to  $r1 - 1$  and for  $j = 0$  to  $c2 - 1$ :

Set  $z[i][j] = 0$ .

Repeat for  $k = 0$  to  $c1 - 1$ : add  $x[i][k] \times y[k][j]$  to  $z[i][j]$ .

**Step 2:** Return to the calling function.

## 3. Test Case Table

| # | Operation         | Input                                                                                                 | Expected Output                                           | Actual Output         | Result |
|---|-------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------|--------|
| 1 | 1. Addition       | $2 \times 3$ , $A = 1\ 2\ 3 / 4\ 5\ 6$ , $B = 7\ 8\ 9 / 10\ 11\ 12$ , $C = 13\ 14\ 15 / 16\ 17\ 18$   | Both sides come out as $21\ 24\ 27 / 30\ 33\ 36$          | Both identical        | PASS   |
| 2 | 1. Addition       | the same three matrices                                                                               | The message says the property is verified                 | Verified              | PASS   |
| 3 | 2. Multiplication | $m\ n\ p\ q = 2\ 3\ 2\ 2$ , $A = 1\ 2\ 3 / 4\ 5\ 6$ , $B = 7\ 8 / 9\ 10 / 11\ 12$ , $C = 1\ 2 / 3\ 4$ | $(A \times B) \times C = 250\ 372 / 601\ 894$             | $250\ 372 / 601\ 894$ | PASS   |
| 4 | 2. Multiplication | the same three matrices                                                                               | $A \times (B \times C)$ is the same $2 \times 2$ matrix   | Both identical        | PASS   |
| 5 | 2. Multiplication | $m\ n\ p\ q = 2\ 2\ 3\ 1$ , so all four sizes differ                                                  | Both sides come out as the $2 \times 1$ matrix $73 / 165$ | $73 / 165$            | PASS   |
| 6 | 1. Addition       | $1 \times 1$ with 5, 6 and 7                                                                          | Both sides come out as 18                                 | 18                    | PASS   |
| 7 | 3. Exit           | choice = 3                                                                                            | Exiting the program, and the loop ends                    | Program ended         | PASS   |

## 4. Source Code

### associative.c

```
/* Experiment 33. Both matrix addition and matrix multiplication are
   associative, so the grouping of the three matrices does not change
   the answer. Variable length arrays are used, because the dimensions
   are only known at run time. */

#include <stdio.h>

void additionAssociative(void);
void multiplicationAssociative(void);

/* The number of columns comes before the matrix, so that the compiler
   knows the shape of each row. */
void readMatrix(int rows, int cols, int m[rows][cols], char name)
{
    int i, j;

    printf("Enter the elements of matrix %c (%d x %d):\n", name, rows, cols);
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            scanf("%d", &m[i][j]);
}

void showMatrix(int rows, int cols, int m[rows][cols], const char *label)
{
    int i, j;

    printf("%s\n", label);
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            if (j > 0)
                printf("\t");
            printf("%d", m[i][j]);
        }
        printf("\n");
    }
}

/* z becomes x times y. The shared side c1 is both the columns of x and
   the rows of y, which is why the product exists at all. */
void multiply(int r1, int c1, int c2, int x[r1][c1], int y[c1][c2],
             int z[r1][c2])
{
    int i, j, k;

    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++) {
            z[i][j] = 0;
            for (k = 0; k < c1; k++)
                z[i][j] += x[i][k] * y[k][j];
        }
}

int same(int rows, int cols, int x[rows][cols], int y[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++)
            if (x[i][j] != y[i][j])
                return 0;

    return 1;
}

int main(void)
{
    int choice;
```

```

do {
    printf("\nMENU\n");
    printf("1. Verify the associative property of matrix addition\n");
    printf("2. Verify the associative property of matrix multiplication\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: additionAssociative(); break;
        case 2: multiplicationAssociative(); break;
        case 3: printf("Exiting the program.\n"); break;
    }
} while (choice != 3);

return 0;
}

/* All three matrices are of the same order, because addition is only
defined between matrices of the same size. */
void additionAssociative(void)
{
    int m, n, i, j;

    printf("Enter the values of m and n: ");
    scanf("%d %d", &m, &n);

    int a[m][n], b[m][n], c[m][n], left[m][n], right[m][n];

    readMatrix(m, n, a, 'A');
    readMatrix(m, n, b, 'B');
    readMatrix(m, n, c, 'C');

    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++) {
            left[i][j] = (a[i][j] + b[i][j]) + c[i][j];
            right[i][j] = a[i][j] + (b[i][j] + c[i][j]);
        }

    showMatrix(m, n, left, "(A + B) + C =");
    showMatrix(m, n, right, "A + (B + C) =");

    if (same(m, n, left, right))
        printf("Both sides are equal, so the associative property is verified\n");
    else
        printf("The two sides are not equal\n");
}

/* A is m x n, B is n x p and C is p x q, so A x B is m x p and B x C is
n x q. Both groupings therefore end up as m x q and can be compared. */
void multiplicationAssociative(void)
{
    int m, n, p, q;

    printf("Enter the values of m, n, p and q: ");
    scanf("%d %d %d %d", &m, &n, &p, &q);

    int a[m][n], b[n][p], c[p][q];
    int ab[m][p], bc[n][q], left[m][q], right[m][q];

    readMatrix(m, n, a, 'A');
    readMatrix(n, p, b, 'B');
    readMatrix(p, q, c, 'C');

    multiply(m, n, p, a, b, ab);
    multiply(m, p, q, ab, c, left);

    multiply(n, p, q, b, c, bc);
    multiply(m, n, q, a, bc, right);

    showMatrix(m, q, left, "(A x B) x C =");
    showMatrix(m, q, right, "A x (B x C) =");

    if (same(m, q, left, right))

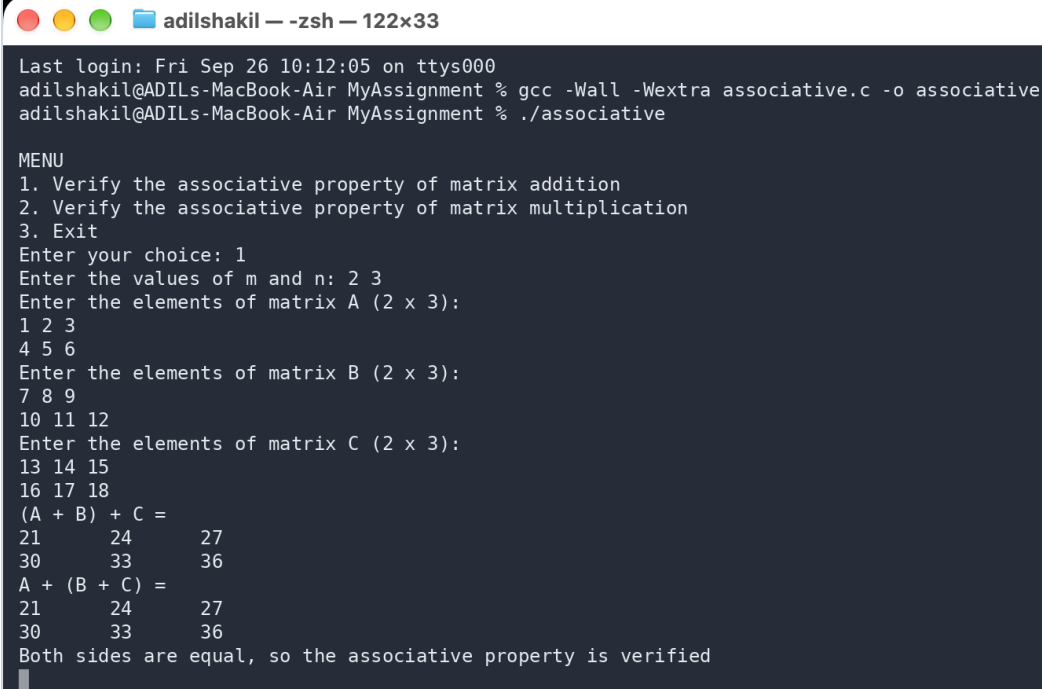
```

```
        printf("Both sides are equal, so the associative property is verified\n");
    else
        printf("The two sides are not equal\n");
}
```

## 5. Compilation and Output

I compiled it with `gcc -Wall -Wextra associative.c -o associative` so that even the small warnings would show up, and it compiled clean in the first attempt itself.

### Addition of Three $2 \times 3$ Matrices



```
adilshakil — zsh — 122x33
Last login: Fri Sep 26 10:12:05 on ttys000
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra associative.c -o associative
adilshakil@ADILs-MacBook-Air MyAssignment % ./associative

MENU
1. Verify the associative property of matrix addition
2. Verify the associative property of matrix multiplication
3. Exit
Enter your choice: 1
Enter the values of m and n: 2 3
Enter the elements of matrix A (2 x 3):
1 2 3
4 5 6
Enter the elements of matrix B (2 x 3):
7 8 9
10 11 12
Enter the elements of matrix C (2 x 3):
13 14 15
16 17 18
(A + B) + C =
21    24    27
30    33    36
A + (B + C) =
21    24    27
30    33    36
Both sides are equal, so the associative property is verified
█
```

## Multiplication with m n p q = 2 3 2 2

```
adilshakil — -zsh — 122x33

MENU
1. Verify the associative property of matrix addition
2. Verify the associative property of matrix multiplication
3. Exit
Enter your choice: 2
Enter the values of m, n, p and q: 2 3 2 2
Enter the elements of matrix A (2 x 3):
1 2 3
4 5 6
Enter the elements of matrix B (3 x 2):
7 8
9 10
11 12
Enter the elements of matrix C (2 x 2):
1 2
3 4
(A x B) x C =
250    372
601    894
A x (B x C) =
250    372
601    894
Both sides are equal, so the associative property is verified
█
```

## Multiplication with All Four Sizes Different, and Exit

```
adilshakil — -zsh — 122x33

MENU
1. Verify the associative property of matrix addition
2. Verify the associative property of matrix multiplication
3. Exit
Enter your choice: 2
Enter the values of m, n, p and q: 2 2 3 1
Enter the elements of matrix A (2 x 2):
1 2
3 4
Enter the elements of matrix B (2 x 3):
1 0 2
0 1 3
Enter the elements of matrix C (3 x 1):
5
6
7
(A x B) x C =
73
165
A x (B x C) =
73
165
Both sides are equal, so the associative property is verified

MENU
1. Verify the associative property of matrix addition
2. Verify the associative property of matrix multiplication
3. Exit
Enter your choice: 3
Exiting the program.
adilshakil@ADILs-MacBook-Air MyAssignment %
█
```

All seven test cases came out right. I worked the  $2 \times 3$  case out by hand as well:  $A \times B$  is 58 64 / 139 154 and multiplying that by C gives 250 372 / 601 894, while  $B \times C$  is 31 46 / 39 58 / 47 70 and A times that gives the

same answer, which is what the program printed.

The sizes were the part I had to be careful with in option 2.  $A$  is  $m \times n$ ,  $B$  is  $n \times p$  and  $C$  is  $p \times q$ , so  $A \times B$  comes out as  $m \times p$  and  $B \times C$  as  $n \times q$ . The two are not even the same shape as each other, but after the third multiplication both groupings land on  $m \times q$ , which is why they can be compared at all. I ran one case with all four values different, 2 2 3 1, to make sure the code was not quietly relying on any of them being equal.